

RAG Ingestion

A production-grade Retrieval-Augmented Generation (RAG) ingestion pipeline built using Clean Architecture principles.

Features

- Unified document parsing using Docling
- Recursive text chunking
- Sentence Transformers embeddings
- ChromaDB vector storage
- Deterministic metadata generation
- Structured logging with structlog
- Dependency injection via a single composition root
- Fully configurable using environment variables
- Unit and integration tests

Architecture

```
Domain
  ↓
Application
  ↓
Infrastructure
  ↓
Interface (CLI)
```

Pipeline

```
Loader
  ↓
Docling Parser
  ↓
Normalizer
  ↓
Chunker
  ↓
Metadata Enrichment
  ↓
SentenceTransformer Embedder
```

ChromaDB Vector Store

Project Layout

```
rag-ingestion/  
├── pyproject.toml  
├── README.md  
├── .env.example  
├── src/  
│   ├── rag_ingestion/  
│   │   ├── application/  
│   │   ├── domain/  
│   │   ├── infrastructure/  
│   │   ├── interface/  
│   │   ├── bootstrap.py  
│   │   ├── config.py  
│   │   └── logging.py  
└── tests/  
    ├── integration/  
    └── unit/
```

Requirements

- Python 3.11+
- uv
- Git

Installation

```
uv sync
```

Running

```
uv run rag-ingestion
```

Running Tests

```
uv run pytest
```

Formatting

```
uv run ruff check .  
uv run ruff format .
```

Type Checking

```
uv run mypy src
```

Configuration

Configuration is provided exclusively through environment variables loaded using `pydantic-settings`.

Copy the example configuration before running the application.

```
cp .env.example .env
```

Initial Supported Formats

- PDF
- DOCX
- TXT
- Markdown
- HTML

All document parsing is delegated to Docling.

Roadmap

- PPTX support
- XLSX support
- CSV support
- Email ingestion
- Hybrid retrieval
- Reranking
- Additional vector store adapters
- Additional embedding providers

License

MIT